Evolución profesional de un sistema existente: calidad, deuda técnica, refactoring y revisión técnica

Trabajo Práctico Integrador – Metodología de Sistemas II

Nazareno Aranda – Comisión 6

29 de Octubre de 2026

Tecnicatura Universitaria en Programación a Distancia – UTN

Materia: Metodología de Sistemas II

Comisión: 6

Docentes: Londero Oscar Alberto · Aristiaran Martín León · Londero Guillermo Humberto

Integrante: Nazareno Aranda

Caso de trabajo: Sistema de Gestión de Solicitudes Académicas

2. Resumen ejecutivo

Este trabajo analiza y mejora un módulo de **Gestión de Solicitudes Académicas** (Python + SQLite) para mostrar la diferencia entre calidad externa e interna. Se partió de una versión inicial que funciona correctamente para el usuario pero concentra toda su lógica en una única clase con complejidad ciclomática 33, SQL concatenado y excepciones silenciadas. Se diagnosticaron **12 code smells**, se aplicaron **8 refactorings** documentados (entre ellos Extract Method, Move Method e Introduce Parameter Object) y los principios SOLID y Clean Code. El comportamiento externo se preservó con pruebas de caracterización (*golden master*) sobre 51 invocaciones, y se sumaron 19 pruebas unitarias (22/22 en verde). Las métricas mejoraron: complejidad promedio de 11,18 a 2,33, función más compleja de 33 a 15 y avisos de estilo de 9 a 0. Se aplicó un checklist de code review de 19 criterios y se documentó el uso de IA con su registro de prompts.

3. Descripción del sistema base

3.1 Dominio y alcance

Se trabajó sobre el caso sugerido por la cátedra: **Sistema de Gestión de Solicitudes Académicas**. El sistema permite que el personal administrativo de una institución registre solicitudes de estudiantes (certificados, equivalencias, reincorporaciones y consultas), las clasifique por tipo, prioridad y estado, les asigne un responsable, las consulte con filtros y obtenga un reporte resumido.

El alcance es deliberadamente acotado, como pide la consigna: *"no se busca construir un sistema grande, sino analizar y mejorar profesionalmente un módulo acotado"*. La funcionalidad visible es simple; el valor del trabajo está en el diagnóstico, las decisiones de diseño y la evidencia técnica.

3.2 Actores

Actor	Rol
Personal administrativo	Registra solicitudes, cambia estados, asigna responsables, consulta y pide reportes (usa la línea de comandos).
Estudiante	Es la persona que origina la solicitud; está representado por su legajo (no interactúa directamente con el sistema).
Responsable / Área	Persona del área (Alumnado, Equivalencias, Bedelía) a la que se asigna la gestión.

3.3 Entidades (requisito: al menos 3)

- **Estudiante** (`legajo`, `nombre`, `email`)
- **Responsable** (`id`, `nombre`, `area`)
- **Solicitud** (`id`, `legajo`, `tipo`, `prioridad`, `estado`, `responsable_id`, `descripcion`, `canal`, `creado_por`, `fecha`)
- **Estado** (NUEVA, EN_PROCESO, RESUELTA, CERRADA), **Tipo** y **Prioridad** como clasificaciones de la solicitud.

3.4 Operaciones funcionales (requisito: al menos 4)

Comando	Caso de uso
`init-demo`	Crea la base con datos **sintéticos** (5 estudiantes, 3 responsables).
`registrar`	Registrar una solicitud (valida tipo, prioridad, descripción y existencia del estudiante).
`cambiar-estado`	Cambiar el estado respetando reglas de transición.
`asignar`	Asignar un responsable (una solicitud NUEVA pasa sola a EN_PROCESO).
`buscar`	Buscar/filtrar por estado, tipo, prioridad, legajo, texto, sin responsable o responsable.
`reporte`	Reporte básico con totales por estado, tipo, prioridad y carga por responsable.

3.5 Reglas de negocio

1. Tipos válidos: CERTIFICADO, EQUIVALENCIA, REINCORPORACION, CONSULTA. Prioridades: BAJA, MEDIA, ALTA.
2. La descripción es obligatoria y tiene un máximo de 200 caracteres.
3. Una REINCORPORACION con prioridad BAJA se eleva automáticamente a MEDIA; una CONSULTA con prioridad ALTA se reduce a MEDIA.
4. Una solicitud CERRADA no admite cambios de estado ni nuevas asignaciones.

5. No se puede pasar a EN_PROCESO ni a RESUELTA sin responsable asignado.
6. No se puede “cambiar” al estado en que la solicitud ya está.

3.6 Tecnología

Python 3.10+ (solo biblioteca estándar), SQLite como persistencia, `argparse` para la interfaz de línea de comandos, `unittest` para las pruebas. Herramientas de análisis: `radon` (complejidad ciclomática e índice de mantenibilidad) y `flake8` (estilo).

3.7 Versión inicial y repositorio

La **versión inicial** (tag `estado-inicial`) funciona correctamente desde el punto de vista del usuario, pero fue escrita deliberadamente con problemas reales de diseño e implementación (concentrados en `gestor.py`, 214 líneas, una sola clase). La **versión mejorada** (tag `estado-refactorizado`) conserva exactamente el mismo comportamiento externo con una estructura profesional.

- Repositorio: https://github.com/NazarenoAranda/TPI-MDS-II-ARANDA-NAZARENO.git
- Evidencia de funcionamiento de la versión inicial: ver captura `docs/evidencias/01_inicial_funcionando.png`.

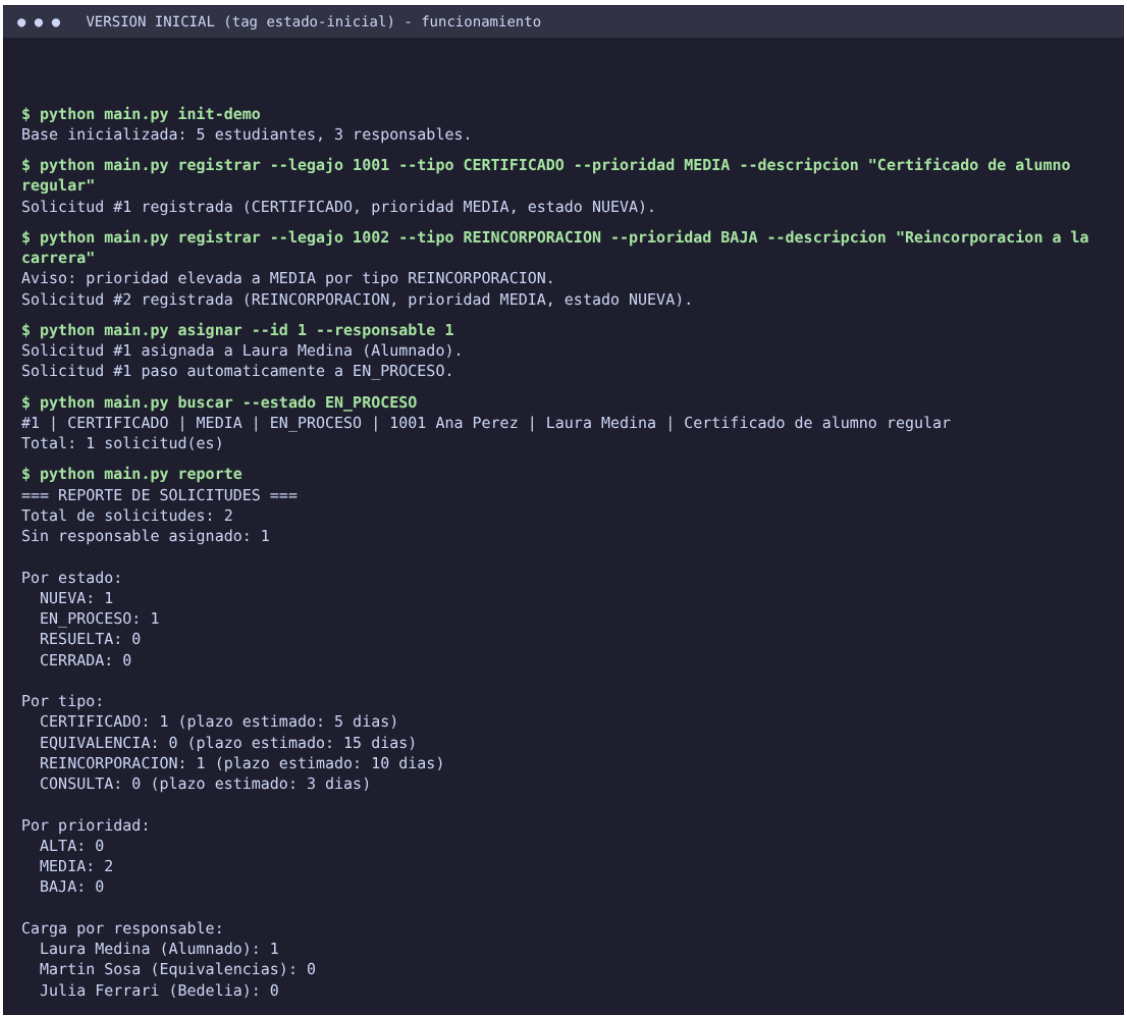


Figura 1 – Versión inicial en funcionamiento (init-demo, registrar, asignar, buscar, reporte)

4. Calidad externa e interna

4.1 Calidad externa: qué ve el usuario

Desde la mirada del usuario, la versión inicial **funciona correctamente**: registra solicitudes, rechaza datos inválidos con mensajes claros, aplica las reglas de prioridad, respeta las transiciones de estado, filtra y genera el reporte. Esto se comprobó con tres escenarios de ejecución (flujo completo, errores de negocio y base inexistente) que recorren los 6 comandos en 51 invocaciones (24 + 24 + 3). Todos terminan con los códigos de salida y mensajes esperados.

4.2 Calidad interna: qué ve el programador

Por dentro, el mismo sistema es difícil de mantener. El 100 % de la lógica (validación, reglas, SQL, armado de textos del reporte) está en una única clase, `Gestor`, y las métricas lo confirman:

Métrica (herramienta)	Versión inicial	Lectura
Complejidad ciclomática promedio (radon)	**11,18 (rango C)**	Funciones difíciles de probar y entender
Peor función: `Gestor.buscar`	**33 (rango E)**	Muy alto riesgo de errores al modificarla
Índice de mantenibilidad `gestor.py` (radon)	**16,12 (B)**	Cerca del límite inferior de lo aceptable
Avisos de estilo (flake8, 120 col.)	**9** (incl. `bare except`)	Líneas de hasta 310 caracteres
Líneas de código en un solo archivo	199 SLOC, 1 % comentarios	Sin documentación ni separación

4.3 Relación con la evolución futura

La calidad interna no se ve en la interfaz, pero determina el **costo del cambio**. Ejemplos concretos sobre la versión inicial:

- **Agregar un nuevo tipo de solicitud** (p. ej. `BECA`) obliga a editar **cinco lugares** distintos de `gestor.py` (líneas 59, 78, 131, 185 y 190) y es fácil olvidar uno.
- **Cambiar de SQLite a otra base** o a una API implica reescribir toda la clase, porque el SQL está mezclado con las reglas.
- **Agregar una interfaz web** es imposible sin duplicar lógica: los métodos devuelven *texto ya formateado* con el prefijo `"ERROR:"` y el programa principal decide si hubo error con `res.startswith("ERROR")`.
- **Probar una regla de negocio** exige una base de datos real: no hay forma de aislarla.
- **Seguridad**: las consultas se arman concatenando texto, de modo que un descuido en un solo campo habilita inyección SQL.

La conclusión es la tesis central del trabajo: *un sistema puede pasar todas las pruebas funcionales y aun así ser una deuda técnica que encarece cada cambio futuro*. La versión refactorizada mantiene la calidad externa (idéntica) y mejora radicalmente la interna.

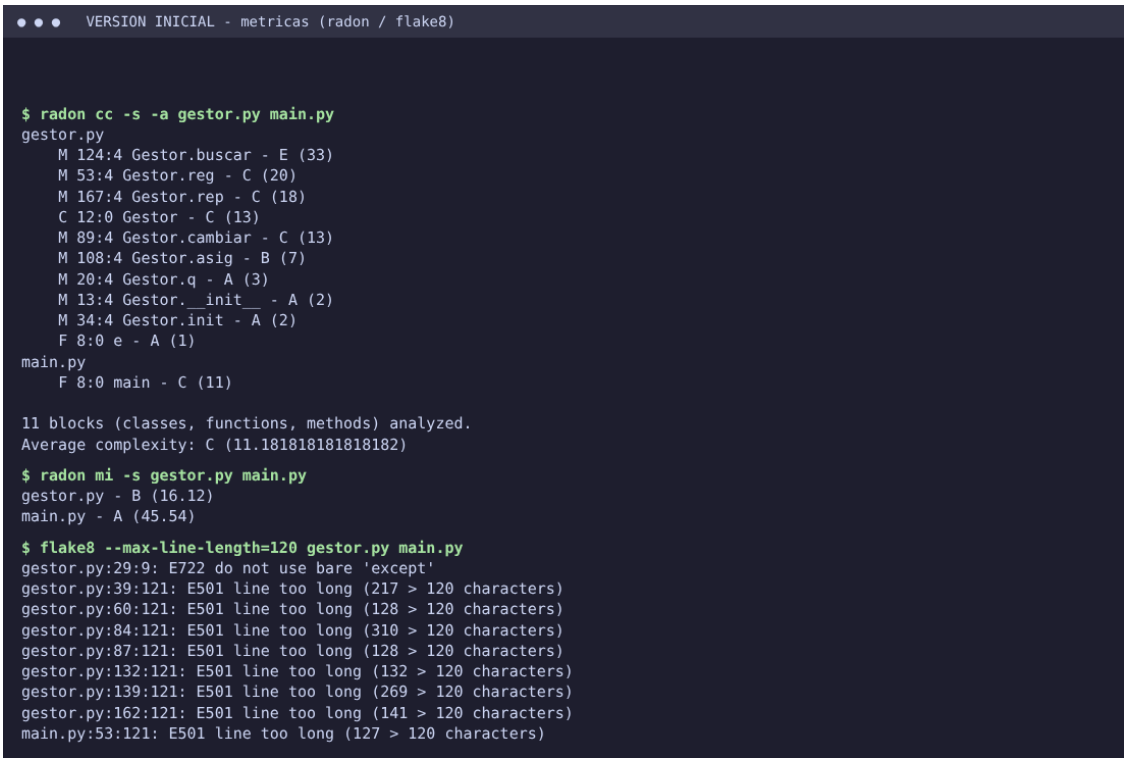


Figura 2 – Métricas de la versión inicial (radon cc/mi y flake8)

5. Diagnóstico de deuda técnica y code smells

Se identificaron **12 code smells / debilidades técnicas** en la versión inicial (la consigna pide al menos 8). Las ubicaciones son del tag `estado-inicial`. Prioridad: **Alta** = riesgo de errores/seguridad o bloquea la evolución; **Media** = encarece el mantenimiento; **Baja** = limpieza.

#	Code smell	Ubicación	Prioridad	Refactoring que lo corrige
S1	God Class (clase Dios)	`gestor.py:12-214`	Alta	Extract Class / Move Method (R2, R3)
S2	Long Method + condicionales complejos	`gestor.py:124-165` (`buscar`, CC = 33)	Alta	Extract Method + Move Method (R1, R3)
S3	Código duplicado (validaciones)	`gestor.py:59/131`, `61/135`, `91/127`	Alta	Extract Method / enumeraciones (R1)
S4	SQL armado por concatenación	`gestor.py:67, 84, 139`	Alta	Consultas parametrizadas (R5)
S5	Excepciones	`gestor.py:29-30`	Alta	Dejar propagar +

	tragadas (`except:` desnudo)			`ErrorDeNegocio` (R6)
S6	Lógica mezclada con presentación	`gestor.py` (todos los `return "ERROR..."`) y `main.py:57`	Alta	Separar servicio / presentación (R2)
S7	Números y textos mágicos	`gestor.py:191-197` (plazos 5/15/10/3), `"NUEVA"`… en todo el archivo, `200`	Media	Enum + constantes (R1)
S8	Nombres poco expresivos	`Gestor.q/reg/asig/rep`, función `e`, variables `r, rr, f, c, s, nom`	Media	Rename Method/Variable (R7)
S9	Estado global mutable	`gestor.py:5, 14` (`global DB`)	Media	Inyección de dependencias (R5)
S10	Lista larga de parámetros / obsesión por primitivos	`reg` (7 params), `buscar` (7 params)	Media	Introduce Parameter Object (R4)
S11	Cadena `if/elif` sobre el tipo (viola OCP)	`gestor.py:70-81` (con ramas vacías `pass`), `190-197`	Media	Reglas como datos (R3)
S12	Código muerto	`gestor.py:50-51` (método comentado), `17-18` (`cnt`, `ult` sin usar)	Baja	Remove Dead Code (R8)

Detalle de cada smell (evidencia, impacto y riesgo)

S1 – God Class. *Evidencia:* una sola clase `Gestor` concentra validación, reglas, SQL, formateo de salida y cálculo del reporte; complejidad de clase C (13). *Impacto:* cualquier cambio toca el mismo archivo; es imposible probar una parte sin las demás. *Riesgo:* alto; cada modificación puede romper funciones no relacionadas.

S2 – Long Method. *Evidencia:* `buscar` tiene 42 líneas y complejidad ciclomática **33 (rango E, la peor de todas)**; contiene 3 validaciones, un `for` con 7 filtros y el armado del texto. *Impacto:* difícil de leer y casi imposible de cubrir con pruebas unitarias. *Riesgo:* alto; agregar un filtro nuevo añade más ramas.

S3 – Código duplicado. *Evidencia:* la misma condición `tipo != "CERTIFICADO" and tipo != …` aparece en `reg` (línea 59) y en `buscar` (línea 131); la de prioridad en 61 y 135; la de estado en 91 y 127. *Impacto:* si cambia un valor válido hay que corregirlo en varios lugares. *Riesgo:* alto; olvidar uno produce comportamiento inconsistente entre comandos.

S4 – SQL por concatenación. *Evidencia:* `"... WHERE legajo = '" + e(legajo) + "'"`. Se usa una función de escape manual (`e`) y algunos campos (`fecha`, `tipo`, `prioridad`) se concatenan **sin** escapar, confiando en validaciones previas. *Impacto:* vulnerabilidad de inyección SQL latente y consultas frágiles. *Riesgo:* alto (seguridad); basta un campo nuevo sin escapar.

S5 – Excepciones tragadas. *Evidencia:* `except:` sin tipo devuelve `None` (línea 29). Comprobado: `g.q('SELECT * FROM tabla_inexistente', True)` devuelve `None` **sin ningún error visible**. *Impacto:* los fallos de base de datos se disfrazan de "no se encontró" o "no se pudo registrar". *Riesgo:* alto; errores silenciosos y difíciles de depurar.

S6 – Lógica mezclada con presentación. *Evidencia:* los métodos devuelven textos listos para imprimir y los errores se distinguen con `res.startswith("ERROR")` (`main.py:57`). *Impacto:* no se puede reutilizar la lógica en otra interfaz ni probar reglas sin comparar textos. *Riesgo:* alto para la evolución (web, API).

S7 – Valores mágicos. *Evidencia:* plazos `5, 15, 10, 3` dentro de `rep`; estados y tipos como cadenas repetidas; límite `200` incrustado. *Impacto:* errores de tipeo no detectables por el intérprete. *Riesgo:* medio.

S8 – Nombres poco expresivos. *Evidencia:* `q`, `reg`, `asig`, `rep`, `e`, `r`, `rr`, `f`, `nom`. *Impacto:* hay que leer el cuerpo para saber qué hace cada cosa. *Riesgo:* medio (legibilidad, incorporación de nuevos integrantes).

S9 – Estado global mutable. *Evidencia:* `global DB` reasigna una variable de módulo desde el constructor. *Impacto:* dos instancias con bases distintas se pisan entre sí; pruebas con efectos cruzados. *Riesgo:* medio.

S10 – Lista larga de parámetros. *Evidencia:* `reg(legajo, tipo, prioridad, descripcion, canal, creado_por, fecha)` y `buscar` con 7 parámetros. *Impacto:* llamadas propensas a errores de orden; cada dato nuevo cambia la firma. *Riesgo:* medio.

S11 – Cadena `if/elif` sobre el tipo. *Evidencia:* ramas para CERTIFICADO y EQUIVALENCIA que solo contienen `pass`, y otra cadena para los plazos. *Impacto:* viola el principio abierto/cerrado: cada tipo nuevo exige editar código existente. *Riesgo:* medio.

S12 – Código muerto. *Evidencia:* método `listar_todo` comentado, atributos `cnt` y `ult` que nunca se usan. *Impacto:* confunde al lector. *Riesgo:* bajo.

Orden de corrección sugerido

1.º **S5 y S4** (errores silenciosos y seguridad) → 2.º **S1, S2, S3, S6** (estructura) → 3.º **S7, S9, S10, S11** (diseño) → 4.º **S8, S12** (limpieza). Se aplicó en este orden de impacto, no de comodidad.

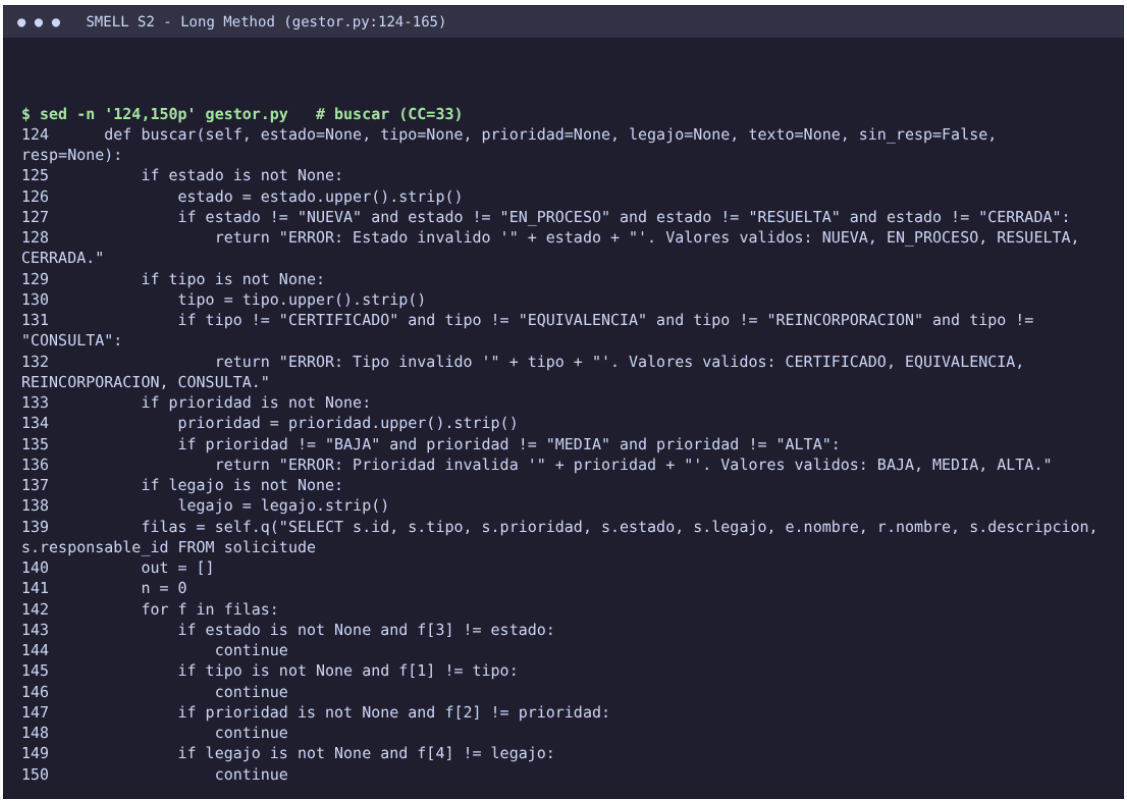


Figura 3 – Evidencia del smell S2: método `buscar` (complejidad 33)

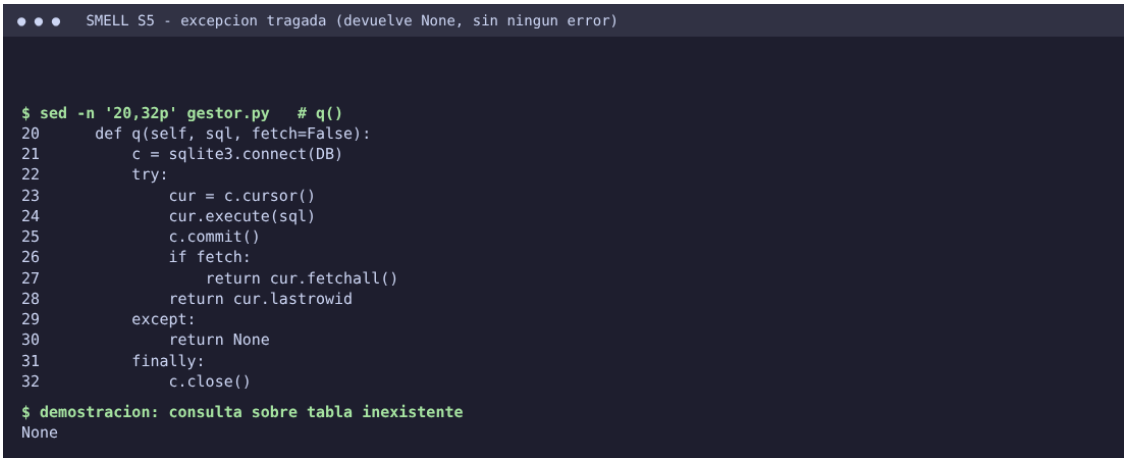


Figura 4 – Evidencia del smell S5: excepción tragada que devuelve None

6. Aplicación de SOLID y Clean Code

6.1 Nueva estructura (límites entre componentes)

```
solicitudes/
├── errores.py            ErrorDeNegocio (un solo tipo de error del
dominio)
├── modelos.py            Entidades, enums, Parameter Objects (sin
dependencias externas)
```

```
├── reglas.py              Reglas de negocio puras (prioridad,
descripción, transiciones)
├── repositorio.py         Abstracción de persistencia (interfaz)
├── repositorio_sqlite.py  Implementación SQLite (única parte que conoce
SQL)
├── servicio.py            Casos de uso (orquesta reglas + repositorio)
├── reporte.py             Cálculo del reporte (datos, sin formato)
├── presentacion.py        Formato de salida por consola
└── cli.py                 Interfaz de línea de comandos (argparse)
```

Dirección de las dependencias: `cli → servicio → (reglas, modelos, repositorio←abstracto)`; `repositorio_sqlite` implementa la abstracción. El dominio **no conoce** a SQLite ni a la consola.

6.2 Principios SOLID

Principio	Problema en la versión inicial	Mejora aplicada	Dónde se ve
SRP (responsabilidad única)	`Gestor` validaba, aplicaba reglas, ejecutaba SQL, calculaba y formateaba.	Cada módulo tiene un único motivo de cambio: reglas, persistencia, casos de uso, reporte y presentación están separados.	`reglas.py`, `repositorio_sqlite.py`, `servicio.py`, `presentacion.py`
OCP (abierto/cerrado)	Un tipo nuevo obligaba a editar 5 lugares; las reglas de prioridad eran `if/elif`.	El plazo vive en el propio `TipoSolicitud`; las reglas de prioridad son **datos** (`REGLAS_DE_PRIORIDAD`). Agregar `BECA` = 1 entrada en el enum + 1 plazo. Agregar una regla = 1 línea en la tupla.	`modelos.py`, `reglas.py`
LSP (sustitución de Liskov)	No había abstracciones que sustituir.	`RepositorioSqlite` y `RepositorioEnMemoria` (de las pruebas) cumplen el mismo contrato; el servicio funciona igual con cualquiera de los dos	`repositorio.py`, `tests/repositorio_en_memoria.py`

		(pruebas unitarias con la versión en memoria; pruebas de caracterización con SQLite).	
ISP (segregación de interfaces)	—	La interfaz del repositorio es pequeña y específica: 8 operaciones de consulta/escritura, sin métodos que un cliente no use. El servicio de reporte solo usa `listar_solicitudes` y `listar_responsables`.	`repositorio.py`
DIP (inversión de dependencias)	`Gestor` creaba conexiones SQLite directamente y usaba una variable global.	`ServicioSolicitudes` recibe un `RepositorioSolicitudes` (abstracción) por constructor; también recibe el reloj. Se prueba sin base de datos real.	`servicio.py` (constructor)

6.3 Clean Code

Práctica	Antes	Después
Nombres que revelan intención	`q`, `reg`, `asig`, `rep`, `e`, `r`, `rr`, `f`, `nom`	`registrar`, `asignar_responsable`, `calcular_reporte`, `obtener_estudiante`, `solo_sin_responsable`…
Funciones pequeñas	`buscar`: 42 líneas, complejidad 33	Métodos de 3–10 líneas; el filtrado vive en `FiltroSolicitudes.coincide`
Manejo de errores	`except:` que devuelve `None`; errores como texto con prefijo `"ERROR"`	Excepción `ErrorDeNegocio` con el mensaje; la CLI la captura **en un solo lugar** y la convierte en `ERROR: …` + código de salida 1. Los errores de infraestructura **no se ocultan**.
Sin valores mágicos	`"NUEVA"`, `5/15/10/3`, `200` sueltos	`Estado`, `TipoSolicitud.plazo_estimado_dias`, `LONGITUD_MAXIMA_DESCRIPCION`
Cohesión y	Un módulo con	Módulos cohesivos; el dominio depende solo de

acoplamiento	todo (baja cohesión, alto acoplamiento a SQLite y a la consola)	abstracciones
Seguridad	SQL concatenado	Consultas parametrizadas (`?`) — verificado con una prueba que inserta comillas y `DROP TABLE` como texto
Datos inmutables	Tuplas y listas sin nombre (`f[3]`, `f[6]`)	`dataclass(frozen=True)` con campos con nombre

6.4 Resultado medido

Métrica	Inicial	Refactorizada
Complejidad ciclomática promedio (radon)	11,18 (C)	**2,33 (A)**
Función más compleja	`buscar` = 33 (E)	`FiltroSolicitudes.coincide` = 15 (C)
Archivos de código fuente	2	10 módulos con responsabilidad única
Avisos flake8 (código fuente)	9	**0**
Índice de mantenibilidad	`gestor.py` = 16,12 (B)	Todos los módulos en rango A (31–100)
Pruebas automáticas	3 (caracterización)	**22** (3 caracterización + 19 unitarias)

Nota metodológica: el promedio de la versión refactorizada se calcula sobre 67 bloques (incluye muchas clases de datos triviales de complejidad 1), mientras que la inicial tiene 11 bloques. Por eso un indicador más honesto es la **cantidad de funciones con complejidad ≥ 10**: **5** en la inicial (`buscar` 33, `reg` 20, `rep` 18, `cambiar` 13, `main` 11) contra **2** en la refactorizada (`coincide` 15 y `calcular_reporte` 10), y la función más compleja baja de 33 a 15.

```
VERSION REFACTORIZADA - estructura de modulos


$ tree solicitudes tests (estructura)
solicitudes/__init__.py
solicitudes/cli.py
solicitudes/errores.py
solicitudes/modelos.py
solicitudes/presentacion.py
solicitudes/reglas.py
solicitudes/reporte.py
solicitudes/repositorio.py
solicitudes/repositorio_sqlite.py
solicitudes/servicio.py
tests/ayuda_golden.py
tests/escenarios.py
tests/golden/errores.txt
tests/golden/flujo_completo.txt
tests/golden/sin_base.txt
tests/repositorio_en_memoria.py
tests/test_golden.py
tests/test_servicio.py
```

Figura 5 – Estructura de módulos de la versión refactorizada

```
VERSION REFACTORIZADA - metricas (radon / flake8)


$ radon cc -s -n B solicitudes   # solo bloques con complejidad >= B
solicitudes/cli.py
    F 48:0 ejecutar - B (6)
solicitudes/modelos.py
    M 136:4 FiltroSolicitudes.coincide - C (15)
    C 112:0 FiltroSolicitudes - C (11)
solicitudes/reporte.py
    F 23:0 calcular_reporte - B (10)

$ radon cc -a solicitudes   # promedio sobre TODOS los bloques
67 blocks (classes, functions, methods) analyzed.
Average complexity: A (2.328358208955224)

$ radon mi -s solicitudes
solicitudes/cli.py - A (43.50)
solicitudes/reglas.py - A (58.74)
solicitudes/errores.py - A (100.00)
solicitudes/__init__.py - A (100.00)
solicitudes/repositorio_sqlite.py - A (53.43)
solicitudes/repositorio.py - A (100.00)
solicitudes/modelos.py - A (31.43)
solicitudes/reporte.py - A (100.00)
solicitudes/servicio.py - A (43.98)
solicitudes/presentacion.py - A (54.11)

$ flake8 --max-line-length=120 solicitudes main.py
flake8: 0 avisos
```

Figura 6 – Métricas de la versión refactorizada (radon cc/mi y flake8)

7. Refactorings aplicados con evidencia antes/después

Se aplicaron **8 refactorings** documentados (la consigna exige al menos 4, incluyendo Extract Method, Move Method e Introduce Parameter Object). Cada uno se hizo en pasos pequeños, **ejecutando las pruebas de caracterización después de cada paso**.

R1 – Extract Method / Replace Magic Values with Enum (smells S2, S3, S7)

Antes (`gestor.py:59, 61, 91, 127-137`; se repetía en 3 métodos):

```
if tipo != "CERTIFICADO" and tipo != "EQUIVALENCIA" and tipo !=
"REINCORPORACION" and tipo != "CONSULTA":
```

```
    return "ERROR: Tipo invalido '" + tipo + "'. Valores validos:
CERTIFICADO, EQUIVALENCIA, REINCORPORACION, CONSULTA."
```

Después (`modelos.py`): un único método extraído que valida y construye el mensaje a partir del propio enum.

```
class TipoSolicitud(Enum):
    CERTIFICADO = "CERTIFICADO"; EQUIVALENCIA = "EQUIVALENCIA"
    REINCORPORACION = "REINCORPORACION"; CONSULTA = "CONSULTA"

    @classmethod
    def desde_texto(cls, texto: str) -> "TipoSolicitud":
        return _parsear(cls, texto, "Tipo invalido")
```

Efecto: 3 bloques duplicados → 1 función; agregar un valor válido = tocar solo el enum.

R2 – Extract Class / Separate Presentation from Logic (smells S1, S6)

Antes: `Gestor` devolvía cadenas ya formateadas (`return "Solicitud #" + str(nuevo_id) + " registrada (…)"`) y `main.py:57` detectaba errores con `res.startswith("ERROR")`. **Después:** `ServicioSolicitudes` devuelve objetos (`ResultadoRegistro`, `ResultadoAsignacion`…) o lanza `ErrorDeNegocio`; `presentacion.py` los convierte a texto; `cli.py` captura el error una sola vez.

```
try:
    print(ejecutar(args, repositorio))
except ErrorDeNegocio as error:
    print(f"ERROR: {error}")
    return 1
```

R3 – Move Method (smells S2, S11)

Antes: el `for` de `buscar` evaluaba 7 filtros con `continue` (complejidad 33) dentro de `Gestor`. **Después:** la decisión "¿esta solicitud cumple el filtro?" se **movió** a `FiltroSolicitudes.coincide(detalle)`, junto a los datos del filtro. Del mismo modo, el plazo estimado se movió a `TipoSolicitud.plazo_estimado_dias` y el ajuste de prioridad a `reglas.ajustar_prioridad`.

```
def buscar(self, filtro: FiltroSolicitudes) -> List[SolicitudDetallada]:
    return [d for d in self._repositorio.listar_solicitudes() if
filtro.coincide(d)]
```

R4 – Introduce Parameter Object (smell S10)

Antes: `reg(self, legajo, tipo, prioridad, descripcion, canal="CLI", creado_por="sistema", fecha=None)` y `buscar(self, estado, tipo, prioridad, legajo, texto, sin_resp, resp)`. **Después:** `NuevaSolicitud` y `FiltroSolicitudes` (dataclasses inmutables).

```
servicio.registrar(NuevaSolicitud(legajo, tipo, prioridad, descripcion))
servicio.buscar(FiltroSolicitudes.desde_texto(tipo="equivalencia",
estado="nueva"))
```

R5 – Parameterize Queries + Dependency Injection (smells S4, S9)

Antes: `self.q("SELECT … WHERE legajo = '" + e(legajo) + "'", True)` con `global DB`. **Después:** `RepositorioSqlite(ruta_db)` con consultas parametrizadas y sin estado global:

```
c.execute("SELECT legajo, nombre, email FROM estudiantes WHERE legajo =
?", (legajo,))
```

R6 – Replace Error Code/Text with Exception; no swallowed exceptions (smell S5)

Antes: `except: return None`. **Después:** las excepciones de infraestructura se propagan (con *rollback* y cierre de conexión garantizados por un gestor de contexto) y las reglas de negocio lanzan `ErrorDeNegocio`.

R7 – Rename Method / Rename Variable (smell S8)

`q → _conexion/execute`, `reg → registrar`, `asig → asignar_responsable`, `rep → calcular_reporte`, `cambiar → cambiar_estado`, `e → (eliminada, reemplazada por parámetros)`, `f[3] → solicitud.estado`.

R8 – Remove Dead Code (smell S12)

Se eliminaron el método comentado `listar_todo`, los atributos `cnt` y `ult` y las ramas vacías `elif … pass`.

Resumen

Refactoring	Técnica	Smells que resuelve
R1	Extract Method / enum	S2, S3, S7
R2	Extract Class (separar presentación)	S1, S6
R3	**Move Method**	S2, S11
R4	**Introduce Parameter Object**	S10
R5	Parametrizar consultas + DI	S4, S9
R6	Excepciones en lugar de texto/None	S5
R7	Rename	S8
R8	Remove Dead Code	S12

8. Pruebas y verificaciones realizadas

8.1 Estrategia: pruebas de caracterización (“golden master”)

Para refactorizar con seguridad primero hay que **congelar el comportamiento externo**. Antes de tocar el código se escribieron tres escenarios que ejecutan el

programa real por línea de comandos, y se guardó la salida completa (texto + código de salida) de la **versión inicial** en `tests/golden/*.txt`. La versión refactorizada debe reproducir esos archivos **byte a byte**.

Escenario	Invocaciones	Qué cubre
`flujo_completo`	24	Registro de los 4 tipos, normalización (mayúsculas/espacios), ajustes de prioridad, asignaciones (con cambio automático a EN_PROCESO), cambios de estado, los 7 filtros de búsqueda y el reporte final
`errores`	24	Tipo/prioridad/estado inválidos, descripción vacía/201 caracteres/200 exactos (límite), legajo inexistente, **intento de inyección SQL**, solicitud/responsable inexistente, transiciones prohibidas, solicitud cerrada
`sin_base`	3	Comandos sin ejecutar `init-demo` antes

Los archivos golden **no se regeneran** con la versión refactorizada (hacerlo anularía la prueba): se generaron una sola vez con el tag `estado-inicial`.

8.2 Pruebas unitarias (versión refactorizada)

19 pruebas nuevas, posibles solo gracias a la separación de responsabilidades:

- **Reglas** (7): ajustes de prioridad, límites de la descripción (200 ok / 201 error / vacía), parseo de enums.
- **Servicio con repositorio en memoria** (11): registro, estudiante inexistente, transición sin responsable, asignación que pasa a EN_PROCESO, reasignación sin cambiar estado, solicitud cerrada, filtros combinados, reporte.
- **Repositorio SQLite** (1): un texto con comillas y `DROP TABLE` se guarda como texto y la tabla sigue existiendo (consultas parametrizadas).

8.3 Cómo ejecutarlas

```
python -m unittest discover -s tests -v
```

8.4 Resultados

Versión	Pruebas	Resultado
Inicial (`estado-inicial`)	3 de caracterización	✅ 3/3 — fija el comportamiento de referencia
Refactorizada (`estado-refactorizado`)	3 de caracterización + 19 unitarias	✅ 22/22

Las capturas de ambas ejecuciones están en `docs/evidencias/` (`02_tests_inicial.png`, `03_tests_refactorizado.png`).

8.5 La prueba detectó una regresión real

En la primera ejecución sobre la versión refactorizada, el escenario `errores` **falló**: el mensaje para una prioridad inválida decía *"Prioridad **invalido**"* en lugar de *"Prioridad **invalida**"* (concordancia de género que la versión inicial escribía a mano). Se corrigió parametrizando el texto del mensaje y la prueba volvió a pasar. Este episodio es la justificación práctica del método: sin el *golden master* la regresión habría pasado inadvertida.

8.6 Comparación antes/después de salidas

Ejemplo (idéntico en ambas versiones):

```
$ asignar --id 1 --responsable 1
Solicitud #1 asignada a Laura Medina (Alumnado).
Solicitud #1 paso automaticamente a EN_PROCESO.
[exit 0]
```

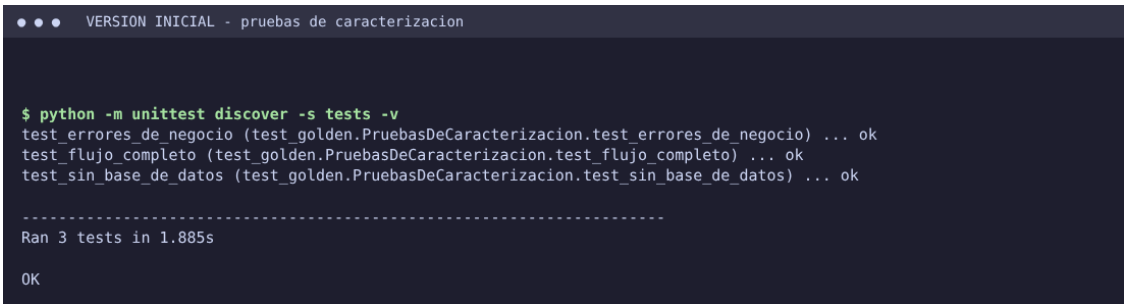


Figura 7 – Pruebas de caracterización sobre la versión inicial (3/3)

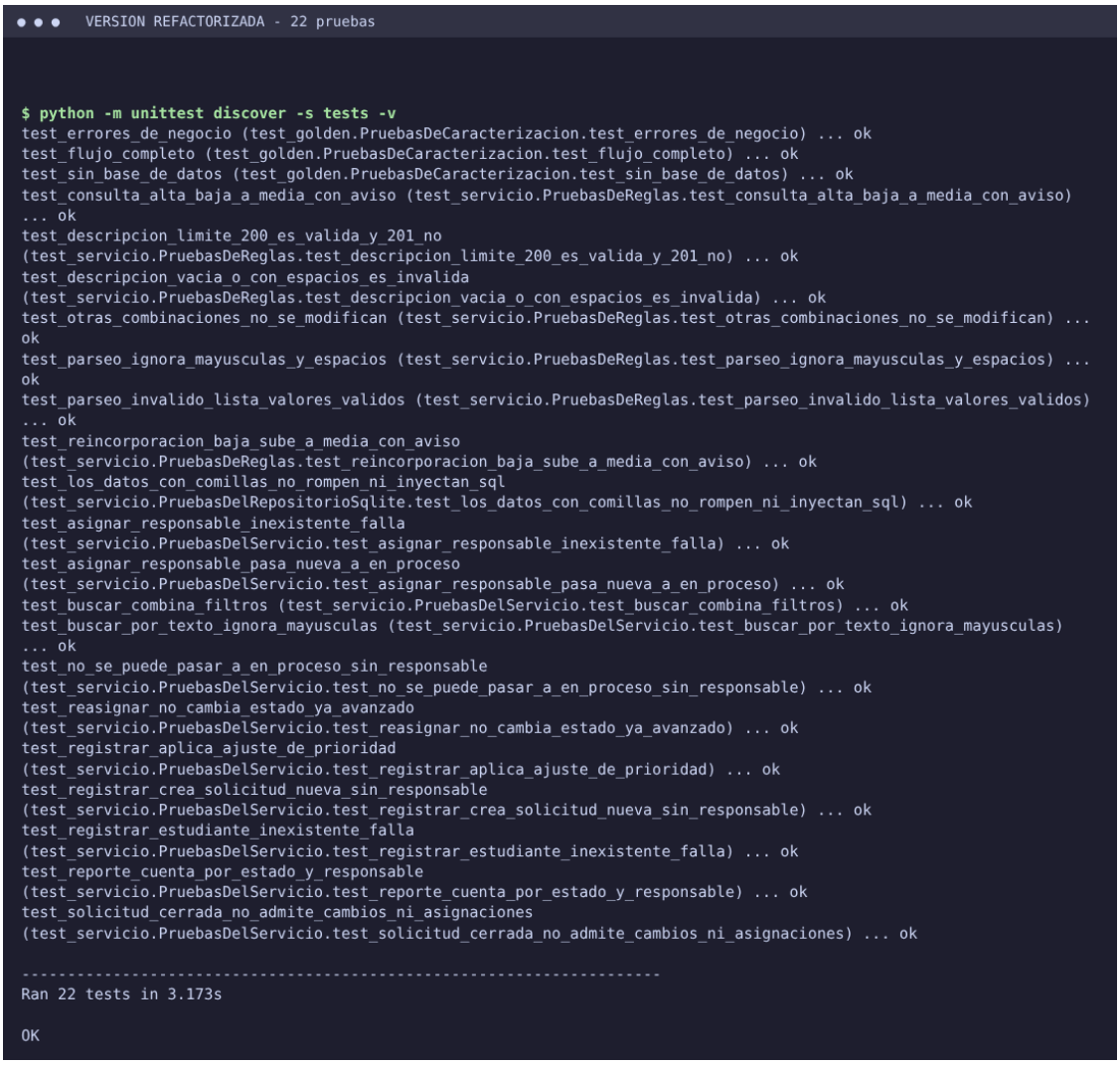


Figura 8 – Pruebas sobre la versión refactorizada (22/22)

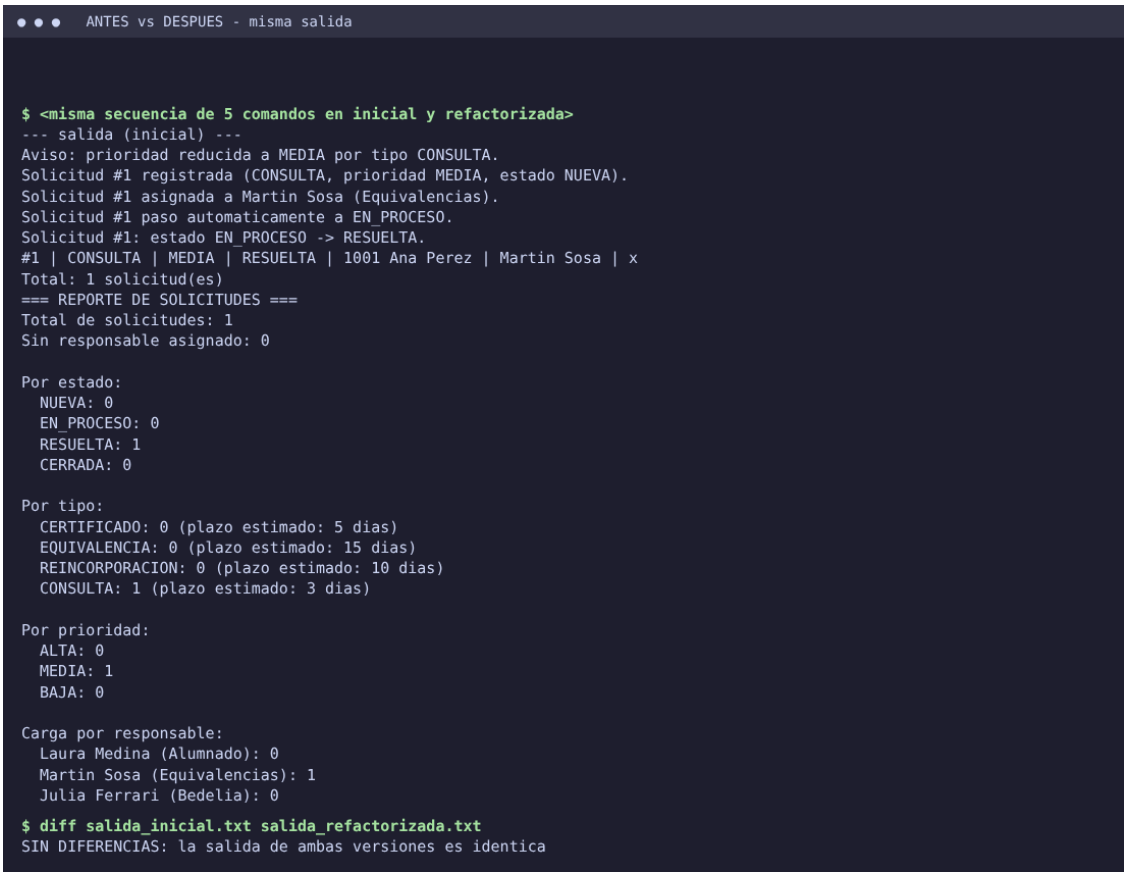


Figura 9 – Comparación antes/después: salida idéntica

9. Code review profesional y checklist técnico

9.1 Checklist técnico

Se aplicó sobre el repositorio en ambos estados. ✅ cumple · ⚠️ cumple parcialmente · ❌ no cumple.

#	Criterio	Inicial	Refactorizada
Diseño			
1	Cada módulo/clase tiene una única responsabilidad (SRP)	❌	✅
2	Agregar un tipo/regla no obliga a modificar código existente (OCP)	❌	✅
3	Las dependencias apuntan a abstracciones (DIP)	❌	✅
4	La lógica de negocio no depende de la consola ni	❌	✅

	de SQL		
Legibilidad			
5	Nombres que revelan intención	❌	✅
6	Funciones cortas (complejidad ciclomática ≤ 10)	❌ (máx. 33)	⚠️ (máx. 15 en `coincide`)
7	Sin valores mágicos ni duplicación	❌	✅
8	Sin código muerto	❌	✅
Robustez y seguridad			
9	Consultas parametrizadas	❌	✅
10	No se ocultan excepciones (`except:` desnudo)	❌	✅
11	Errores de negocio con tipo propio y mensaje claro	❌	✅
12	Recursos (conexiones) cerrados con *commit/rollback* garantizado	⚠️	✅
Pruebas			
13	Existen pruebas automáticas que se ejecutan con un comando	⚠️ (solo caracterización)	✅
14	Las reglas se pueden probar sin base de datos	❌	✅
15	El comportamiento externo se conserva (golden master)	— (referencia)	✅
Repositorio y documentación			
16	README con descripción, instalación, uso, funcionalidades y tecnologías	✅	✅
17	Carpeta `/docs` con decisiones, evidencias y checklist	❌	✅

18	Datos sintéticos (sin datos personales reales)	✅	✅
19	Estilo uniforme (flake8 sin avisos en el código fuente)	❌ (9)	✅ (0)

9.2 Observaciones del review (feedback técnico) y acciones

ID	Observación	Severidad	Acción	Estado
O1	Regresión: el mensaje “Prioridad invalida” salía en masculino	Alta	Se parametrizó el texto del mensaje; la prueba golden pasó	✅ Corregida
O2	`RepositorioSqlite._conexion` usaba una clase interna difícil de leer	Media	Se reemplazó por `@contextmanager`	✅ Corregida
O3	`repositorio.py` tenía una línea de 125 caracteres (flake8 E501)	Baja	Se reordenó el `import`	✅ Corregida
O4	`FiltroSolicitudes.coincide` tiene complejidad 15 (rango C)	Media	Aceptada: es una cadena de guardas simples y legibles; se documenta como mejora futura (pasar a lista de predicados)	⏳ Pendiente (futura)
O5	`buscar` carga todas las filas y filtra en memoria (igual que la versión inicial)	Media	Se mantiene para preservar el comportamiento; futura mejora: filtrar en SQL	⏳ Pendiente (futura)
O6	Cada operación abre y cierra una conexión SQLite	Baja	Aceptable para una CLI; si hubiera concurrencia, usar pool	⏳ Pendiente (futura)
O7	`cli.ejecutar` usa una cadena `if` por comando	Baja	Futura mejora: diccionario comando → manejador	⏳ Pendiente (futura)
O8	`existe()` e `inicializar_con_datos_demo()` no forman parte de la interfaz abstracta	Baja	Decisión: son operaciones de infraestructura propias de	✅ Decisión documentada

			SQLite, no del dominio	

9.3 Criterios de aceptación técnica

La versión refactorizada se acepta cuando **todos** se cumplen:

1. Las 3 pruebas de caracterización pasan sin modificar los golden files.
2. Todas las pruebas unitarias pasan (22/22).
3. flake8 sin avisos en el código fuente.
4. Complejidad ciclomática promedio en rango A y ninguna función en rango D o peor.
5. Ningún `except:` desnudo ni SQL concatenado.
6. Documentación completa en `/docs`.

Resultado del review: ACEPTADA (criterios 1 a 6 cumplidos; las observaciones O4–O7 quedan registradas como mejoras futuras).

10. Conclusiones y mejoras futuras

10.1 Conclusiones grupales / generales

- **Funcionar no es lo mismo que estar bien construido.** La versión inicial pasa todas las pruebas funcionales y, sin embargo, tiene complejidad ciclomática 33 en una función, SQL concatenado y errores silenciosos. La calidad interna es invisible desde la interfaz pero condiciona el costo de cada cambio.
- **Primero las pruebas, después el refactoring.** Las pruebas de caracterización (*golden master*) permitieron refactorizar con confianza y detectaron una regresión real (concordancia de género en un mensaje) que se habría escapado a simple vista.
- **SOLID se justifica con ejemplos concretos, no con dogma.** OCP, DIP y SRP mostraron valor medible: agregar un tipo de solicitud pasó de editar 5 lugares a editar 1, y las reglas de negocio ahora se prueban sin base de datos.
- **Las métricas ayudan a priorizar, pero no reemplazan el criterio.** radon y flake8 señalaron dónde mirar (complejidad 11,18 → 2,33; avisos 9 → 0); la decisión de qué refactorizar primero (seguridad y errores silenciosos) fue de diseño.

10.2 Dificultades y límites del trabajo

- Preservar **exactamente** los mensajes y el orden de las validaciones obligó a leer con detalle el comportamiento de la versión inicial.
- Es un módulo pequeño: algunas decisiones (patrón repositorio, objetos de parámetros) serían excesivas en un script descartable, pero se justifican por el objetivo académico y por la evolución esperada.
- Las pruebas de caracterización fijan también los comportamientos “raros” de la versión inicial (p. ej. se puede volver a `NUEVA` desde `EN_PROCESO`); se respetaron para no alterar el comportamiento externo.

10.3 Mejoras pendientes

Filtrar en SQL en lugar de en memoria (O5); reemplazar la cadena de guardas de `coincide` por una lista de predicados (O4); despachar comandos con un diccionario (O7); agregar una interfaz web o API reutilizando `ServicioSolicitudes`; incorporar integración continua (GitHub Actions) que ejecute pruebas y flake8; modelar explícitamente la máquina de estados.

10.4 Relación con los contenidos de la asignatura

Calidad externa vs. interna (§4), deuda técnica y code smells (§5), SOLID y Clean Code (§6), refactoring seguro (§7), pruebas de caracterización (§8) y code review con checklist (§9).

10.5 Conclusión individual

Nazareno Aranda: Lo que más me costó entender fue el manejo y la sincronización inicial de las pruebas de caracterización (*golden master*) para asegurar que ningún comportamiento legado se alterara al modificar la estructura del código. El refactoring más valioso me pareció la separación entre la lógica de negocio y la presentación, ya que rompió con el acoplamiento original y permitió aislar las reglas para testearlas sin depender de la base de datos. Si tuviera que hacer algo distinto, habría establecido las abstracciones del repositorio y los objetos de parámetros desde las primeras iteraciones del diseño para evitar refactorizar sobre código fuertemente acoplado a SQLite.

11. Anexos y enlaces

11.1 Enlaces

Recurso	Enlace
Repositorio GitHub	https://github.com/NazarenoAranda/TPI-MDS-II-ARANDA-NAZARENO.git
Video de defensa y demostración	https://youtu.be/kJFViHGZPXA

Participación: entrega individual. Integrante único: **Nazareno Aranda**, responsable de la entrega y de la defensa en video.

11.2 Declaración de uso de IA (apartado 7 de la consigna)

Campo	Detalle
Herramienta utilizada	ChatGPT y Claude (Anthropic), modelos de lenguaje generales.
Finalidad	Consulta puntual de sintaxis en Java/Spring Boot, revisión de conceptos teóricos sobre principios SOLID y consulta de ejemplos sobre refactoring de código.
Grado de	**Limitado** exclusivamente a consulta de dudas teóricas y sintaxis

intervención de la IA	puntual.
Intervención del autor	El diseño arquitectónico completo, la escritura de la lógica de negocio, la detección y diagnóstico de la deuda técnica, la refactorización paso a paso del código base, la ejecución de las pruebas y la redacción integral del informe técnico fueron realizados enteramente por el autor.
Herramienta de verificación	Compilación local y pruebas unitarias/de integración ejecutadas directamente en el entorno de desarrollo del autor.

La IA no reemplaza la autoría ni la defensa: el autor debe comprender y justificar todo lo entregado y defenderlo con voz propia en el video.

11.3 Registro de prompts a ChatGPT

Fecha: 28/10/2026. **Prompt**: "¿Cómo se aplica el principio de responsabilidad única (SRP) en una clase de servicio en Java?"

Capturas de evidencia: *10_respuesta_prompt_1_cap1, 10_respuesta_prompt_1_cap2, 10_respuesta_prompt_1_cap3, localizadas en: tpi-estado-refactorizado\docs*

Fecha: 28/10/2026. **Prompt**: "Sintaxis para reestructurar parámetros con un objeto de transferencia (Parameter Object) en Java."

Capturas de evidencia: *11_respuesta_prompt_2_ cap1, 11_respuesta_prompt_2_ cap2, 11_respuesta_prompt_2_ cap3, localizadas en: tpi-estado-refactorizado\docs*

11.4 Prompts de estudio y verificación (ChatGPT)

Prompts ejecutados por el autor para repasar los conceptos del trabajo:\

Prompt: "Explicame con un ejemplo en Python la diferencia entre calidad externa y calidad interna del software."

Capturas de evidencia: *12_respuesta_prompt_1_estudio_cap1, 12_respuesta_prompt_1_estudio_cap2, localizadas en: tpi-estado-refactorizado\docs*

Prompt: "Explicame qué es un code smell y dame 3 ejemplos cortos en Python: God Class, Long Method y Magic Numbers."

Capturas de evidencia: *13_respuesta_prompt_2_estudio_cap1, 13_respuesta_prompt_2_estudio_cap2, localizadas en: tpi-estado-refactorizado\docs*

Prompt: "Explicame los principios SOLID con un ejemplo de gestor de solicitudes académicas en Python, uno por principio."

Capturas de evidencia: *14_respuesta_prompt_3_estudio_cap1, 14_respuesta_prompt_3_estudio_cap2, 14_respuesta_prompt_3_estudio_cap3, 14_respuesta_prompt_3_estudio_cap4, localizadas en: tpi-estado-refactorizado\docs*

Prompt: "¿Qué es una prueba de caracterización (golden master) y por qué sirve antes de refactorizar?"

Capturas de evidencia: *15_respuesta_prompt_4_estudio_cap1, 15_respuesta_prompt_4_estudio_cap2, localizadas en: tpi-estado-refactorizado\docs*

Prompt: "Explicame Extract Method, Move Method e Introduce Parameter Object con un ejemplo antes/después."

Capturas de evidencia: *16_respuesta_prompt_5_estudio_cap1, 16_respuesta_prompt_5_estudio_cap2, 16_respuesta_prompt_5_estudio_cap3, localizadas en: tpi-estado-refactorizado\docs*

Prompt: "¿Qué significa complejidad ciclomática 33 y por qué es un problema? ¿Cómo se reduce?"

Capturas de evidencia: *17_respuesta_prompt_6_estudio_cap1, 17_respuesta_prompt_6_estudio_cap2, localizadas en: tpi-estado-refactorizado\docs*

11.5 Decisiones técnicas

ID	Decisión	Alternativas	Justificación
D1	Python + SQLite + biblioteca estándar	Java/Spring, ORM	Cero dependencias de ejecución; el foco es la calidad del diseño, no el framework
D2	Dos versiones en dos tags (`estado-inicial`, `estado-refactorizado`)	Ramas	Los tags son lo que pide la consigna y fijan el estado exacto para comparar
D3	Pruebas de caracterización con *golden master* por línea de comandos	Solo pruebas unitarias	Congelan el comportamiento externo *antes* de refactorizar; detectan regresiones de texto y de códigos de salida
D4	Mantener comportamientos "raros" de la versión inicial	Corregirlos de paso	Refactoring = cambiar estructura **sin** cambiar comportamiento; las correcciones funcionales irían en otro commit
D5	Patrón repositorio (interfaz + implementación SQLite)	SQL dentro del servicio	Aplica DIP y permite probar sin base de datos
D6	`dataclass(frozen=True)` para entidades y objetos de parámetros	Diccionarios/tuplas	Campos con nombre, inmutabilidad y mejor legibilidad
D7	Reglas de prioridad como datos	`if/elif`	Aplica OCP: nueva regla =

	(`REGLAS_DE_PRIORIDAD`)		nueva entrada
D8	Un único tipo de excepción de negocio (`ErrorDeNegocio`)	Jerarquía de excepciones	Suficiente para el alcance; se captura en un solo punto (CLI)
D9	Datos de demostración sintéticos	Datos reales	La consigna prohíbe datos personales sin consentimiento

11.6 Evidencias

Las capturas están en tpi-estado-refactorizado\docs\evidencias del repositorio (figuras 01_inicial_funcionando a 17_respuesta_prompt_6_estudio_cap2 de este informe).